

ShengYu Talk

2021-03-04 · C/C++教學

C++ explicit 用法與範例

本篇 ShengYu 介紹 C++ explicit 用法與範例，C++ 裡有隱性轉換 (implicit conversion) 跟顯性轉換 (explicit conversion)，今天來介紹什麼是隱性轉換？什麼是顯性轉換？並且示範一下這兩者的差異。

C++ explicit 基本用法與範例

在 C++ 中 explicit 這個關鍵字最常出現在建構子前面，我們就來介紹這者差異是什麼，如下列範例所示，`MyInteger n1 = 5;` 寫法是數值寫在等號右邊，編譯器會自動幫你去轉換成 `MyInteger(int n)`，這個轉換就是隱性轉換，而 `MyInteger n2(6);` 寫法是很明確地直接呼叫建構子，這個轉換就是顯性轉換，這就是這兩種的差異。

以下這個範例示範支援隱性轉換和顯性轉換的寫法，

cpp-explicit.cpp

```
1 // g++ cpp-explicit.cpp -o a.out
2 #include <iostream>
3 using namespace std;
4
5 class MyInteger {
6 public:
7     MyInteger(int n) : data(n) {
8         cout << "data = " << data << "\n";
9     }
10 private:
```

```
11     int data;  
12 };  
13  
14 int main() {  
15     MyInteger n1 = 5;  
16     MyInteger n2(6);  
17     return 0;  
18 }
```

輸出如下，

```
1 data = 5  
2 data = 6
```

那今天我們想要禁止隱性轉換這種寫法的話只需要在 `MyInteger(int n)` 建構子前面加上 `explicit` 關鍵字，告訴編譯器只能用顯性轉換，那這樣的隱性轉換寫法編譯器就會在編譯時期檢查時就會擋下來，出現編譯錯誤給開發者知道，這樣我們就能夠成功地防止開發者去寫這種程式碼，

```
1 cpp-explicit.cpp: In function 'int main()':  
2 cpp-explicit.cpp:15:20: error: conversion from 'int' to non-scalar type  
3     MyInteger n1 = 5;  
4                     ^
```



接著開發者只能寫 `MyInteger n2(6);` 呼叫建構子 `direct-initialization` 的寫法了，而原本的 `MyInteger n1 = 5;` `copy-initialization` 寫法就不行使用了，如下所示，

cpp-explicit2.cpp

```
1 // g++ cpp-explicit2.cpp -o a.out  
2 #include <iostream>  
3 using namespace std;  
4  
5 class MyInteger {  
6 public:  
7     explicit MyInteger(int n) : data(n) {  
8         cout << "data = " << data << "\n";  
9     }
```

```
10 private:
11     int data;
12 };
13
14 int main() {
15     //MyInteger n1 = 5;
16     MyInteger n2(6);
17     return 0;
18 }
```

輸出如下，

```
1 data = 6
```

C++ explicit 實際範例

剛剛上述已經了解了隱性轉換 implicit conversion 跟顯性轉換 explicit conversion 的差異，但什麼時候會需要用到 explicit 呢？這種情形大概常發生在以下多個建構子情形中，如下列範例所示，有一個 MyString 類別裡有兩種建構子，一種是傳入整數 n 以便動態記憶體配置 n 的大小的 char，另一種是傳入指向字元陣列的指標然後在裡面複製一份，

```
1 // g++ cpp-explicit3.cpp -o a.out
2 #include <iostream>
3 using namespace std;
4
5 class MyString {
6 public:
7     MyString(int n) { // allocate n bytes to the MyString object
8         cout << "MyString(int n)\n";
9     }
10
11     MyString(const char *p) { // initializes object with char *p
12         cout << "MyString(const char *p)\n";
13     }
14 };
15
16 int main() {
17     MyString s1 = 10;
18     MyString s2(10);
19     MyString s3 = "hello world";
```

```
20     MyString s4("123456");
21     return 0;
22 }
```

輸出如下，那我們目前可以接受這四種初始化寫法，以及實際上這四種寫法對應到的哪個建構子，

```
1  MyString(int n)
2  MyString(int n)
3  MyString(const char *p)
4  MyString(const char *p)
```

可是 s1 的寫法讓人容易誤會，跟 s4 很像，為了不讓其他開發者混淆，所以我想避免這種隱式轉換，那麼我就在 MyString(int n) 前加上 explicit，結果就變成下面這樣的例子，為了更好理解這個範例，我特地把程式碼寫的更詳細完整一點，讓我們看看結果會怎樣吧！

```
1  // g++ cpp-explicit4.cpp -o a.out
2  #include <iostream>
3  #include <string.h>
4  using namespace std;
5
6  class MyString {
7  public:
8      explicit MyString(int n) { // allocate n bytes to the MyString object
9          data = new char[n];
10         cout << "MyString(int n)\n";
11     }
12
13     MyString(const char *p) { // initializes object with char *p
14         int len = strlen(p);
15         data = new char[len+1];
16         strcpy(data, p);
17         cout << "MyString(const char *p), data = " << data << "\n";
18     }
19
20     ~MyString() {
21         delete[] data;
22     }
```

```
23 private:
24     char *data;
25 };
26
27 int main() {
28     //MyString s1 = 10;
29     MyString s2(10);
30     MyString s3 = "hello world";
31     MyString s4("123456");
32     return 0;
33 }
```

結果輸出如下，如果再使用 s1 這種 copy-initialization 寫法就會出現編譯錯誤，開發者就不會使用這種寫法了，改完之後剩下這幾種寫法就比較原本的明確多了，

```
1 MyString(int n)
2 MyString(const char *p), data = hello world
3 MyString(const char *p), data = 123456
```

當然實際上可能不只這種情形，但我們了解了 explicit 用法後，我們就能依照當時的狀況選擇要不要使用 explicit 這個關鍵字，以上就是 ShengYu 對 C++ explicit 的簡單用法介紹，希望讓你對 explicit 有更深刻的體會。

其它參考

explicit specifier - cppreference.com

<https://en.cppreference.com/w/cpp/language/explicit>

c++ - What does the explicit keyword mean? - Stack Overflow

<https://stackoverflow.com/questions/121162/what-does-the-explicit-keyword-mean>

其它相關文章推薦

[C/C++ 新手入門教學懶人包](#)

[C/C++ 字串轉數字的4種方法](#)

[C++ virtual 的兩種用法](#)

[C/C++ 字串反轉 reverse](#)

[C/C++ call by value傳值, call by pointer傳址, call by reference傳參考 的差別](#)

[C++ 類別樣板 class template](#)

[std::sort 用法與範例](#)

[std::find 用法與範例](#)

[std::queue 用法與範例](#)

[std::map 用法與範例](#)
[std::deque 用法與範例](#)
[std::vector 用法與範例](#)

#C/C++

 Share

NEWER

[Python 檢查 dict 字典是否為空](#)

OLDER

[Python 取得 ip 的方法](#)

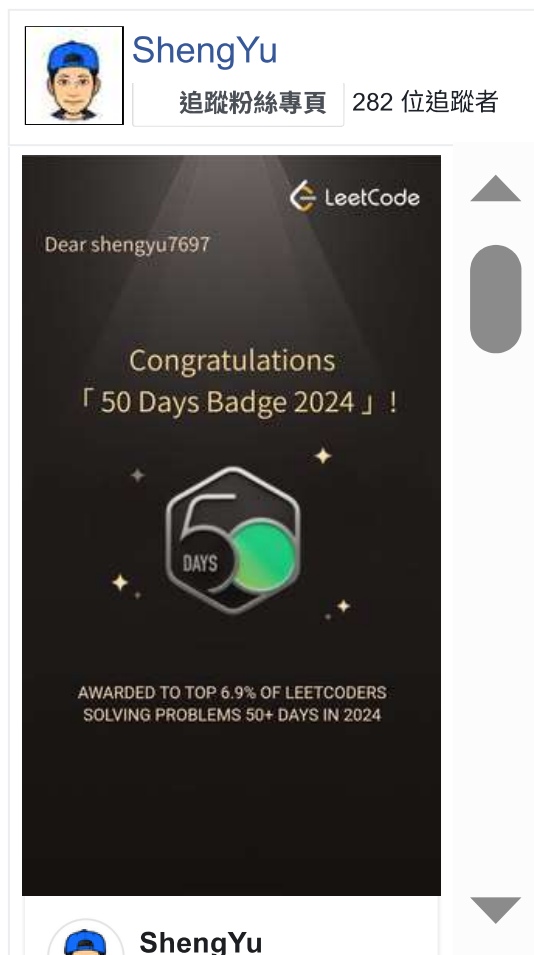
精選文章

[Python 基礎教學目錄](#)
[C/C++ 入門教學目錄](#)
[Linux 常用指令教學懶人包](#)

最新文章

[反模式：常見的不良設計實踐](#)
[軟體開發常見陷阱：技術債累積過多](#)
[軟體開發常見陷阱：過早優化 Premature Optimization](#)
[設計原則 - 依賴反轉原則 Dependency Inversion Principle](#)
[設計原則 - 介面隔離原則 Interface Segregation Principle](#)

粉絲專頁



分類

[Arduino](#) (4)
[C/C++教學](#) (221)
[Github](#) (14)
[Git教學](#) (20)
[OpenCV教學](#) (8)
[PHP教學](#) (14)
[Projects](#) (6)
[Python教學](#) (239)
[Qt教學](#) (7)
[軟體開發](#) (3)